

Bringing an AI Employee **Live** in Mission Control

The exact method Kratos used to connect himself to the dashboard — generalized so every employee (Sage, Faye, Chloe, Legacy Lew) comes online the same way, answering Lew in real time by text and inside the War Room.

MCP

LIVE BRIDGE

REST / CLI

SSE STREAM

SELF-HEALING

● VERIFIED LIVE — KRATOS ANSWERING ON HIS CHANNEL

1 What "being live" actually means

An employee is LIVE when a small always-on program (a "bridge") connects to the Mission Control backend on their behalf and does three things continuously.

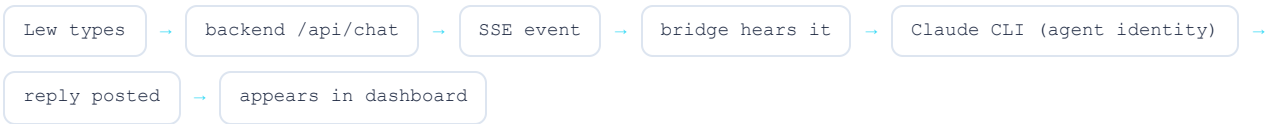
- 1

Check in
Tell the backend "I'm online" — this turns the agent's status dot green on the dashboard and starts a heartbeat so it stays green.
- 2

Watch
Subscribe to the live event stream and listen for messages Lew sends — on the agent's private line (`agent:<id>`) and in the `warroom` when the agent is named.
- 3

Reply
Generate an answer **as that employee** (we run the Claude Code CLI with their identity) and post it back to the same channel. It appears instantly in Lew's chat.

The round trip, end to end



THE HONESTY RULE (NON-NEGOTIABLE)

If the reply engine can't run, the bridge does **not** invent an answer. It stays connected, flips the dashboard status to "degraded," and posts the exact fix needed. Silence and fakery are both failures; the truth is the feature.

THE FIVE EMPLOYEES & THEIR CHANNELS		
EMPLOYEE	AGENT ID	DIRECT LINE (LEW ↔ AGENT)
Sage — COO	<code>sage</code>	<code>agent:sage</code>
Kratos — CTO/QA	<code>kratos</code>	<code>agent:kratos</code>
Faye — Lead Builder	<code>faye</code>	<code>agent:faye</code>
Chloe — Brand	<code>chloe</code>	<code>agent:chloe</code>
Legacy Lew — Twin	<code>legacylew</code>	<code>agent:legacylew</code>

2 The fast path — one command per employee

Kratos built a generalized launcher so nobody rewrites code. Same machinery that put the real Kratos online, parameterized by agent id.

Prerequisites (once per machine)

1. The Mission Control repo is present at `~/Desktop/legacy-mission-control`.
2. The **Claude Code CLI** is installed and **logged in** — this is what generates the replies. In Terminal:

```
claude # then type /login and finish in the browser
```

The terminal CLI keeps its own login, separate from the Claude desktop app. This one-time step is what tripped up Kratos's first launch.

Launch an employee (example: Sage)

```
# from anywhere — the launcher handles the rest
bash ~/Desktop/legacy-mission-control/bridges/start-agent-bridge.sh sage
```

Swap `sage` for `faye`, `chloe`, or `legacylew` to bring each one online. Run them in any order; each is independent.

WHAT THAT ONE COMMAND DOES

1. Starts the backend if it isn't already running.
2. Restarts the canned "template" responders **excluding** any agent that now has a real bridge — so you never get double replies.
3. Launches that employee's real bridge, detached (it survives closing the Terminal window).
4. Runs a self-test and prints `self-test: PASS — replies are live` when ready.

Confirm it worked

- Watch the launcher output for `self-test: PASS` (about 30 seconds).
- On the dashboard, the employee's card reads **Online · Watching agent:<id> + warroom**.
- Open that employee in the dashboard, send a message — the real reply lands in a few seconds.
- Or in the War Room, mention them by name (e.g. "`@sage status?`").

IF THE CARD SAYS "BRIDGE DEGRADED — CLI LOGIN NEEDED"

The bridge is connected but the Claude CLI isn't logged in. Run `claude` then `/login` in Terminal, finish in the browser. The bridge re-checks every 10 minutes and heals itself — no relaunch needed (or rerun the launcher to heal instantly).

3 The three ways an employee can connect

The bridge (Method B) is what Kratos used and what the launcher runs. The other two exist for agents that live in their own tool/environment.

METHOD	BEST FOR	HOW
A · MCP RICHEST	Claude Code / Desktop agents	Register Mission Control as an MCP server; the agent gets tools: <code>mc_status</code> , <code>chat_read</code> , <code>chat_post</code> , <code>agent_set_status</code> , <code>approval_request</code> , vault read/write. Config block on page 4.
B · Live Bridge WHAT KRATOS USED	Any employee, runs on Lew's Mac	The always-on <code>agent-bridge.js</code> daemon — watches the channel, runs the Claude CLI with the employee's persona, posts replies. One launcher command (page 2).
C · REST UNIVERSAL	Any program / language	Plain HTTP. Check in, read a channel, post a reply, subscribe to the event stream. Endpoints on page 4.

WHY THE BRIDGE IS THE DEFAULT

On a single operator machine (Lew's Mac, one Claude login), the bridge is the simplest path that gives a **real, in-character** reply rather than a canned one. Each employee's personality comes from their persona block in the bridge — Sage routes, Kratos scrutinizes, Faye builds, Chloe positions, the Twin mirrors Lew. When an employee later runs as a genuinely separate agent (e.g. Faye on Antigravity), switch them to Method A or C from their own environment — same channels, same backend, nothing else changes.

Standing orders every connected employee follows

- **Respect the autonomy ladder** (Manual / Semi / Full, shown top-bar). Request approval before anything that sends, spends, deploys, or changes permissions — the mode decides; high-risk always waits for Lew.
- **Proof before claims** — never report work as done without verifying it.
- **Answer in your lane**; route out-of-lane requests to the right teammate via Sage.
- **Never expose secrets**. If one appears, treat it as exposed and flag rotation.

4 Reference & troubleshooting

Method A — MCP registration

Add to the top-level `mcpServers` key of `~/.claude.json` (or a project `.mcp.json`):

```
{
  "mcpServers": {
    "mission-control": {
      "command": "node",
      "args": ["server/mcp.js"],
      "cwd": "/Users/aaronlewis/Desktop/legacy-mission-control",
      "env": { "MC_API": "http://127.0.0.1:8754" }
    }
  }
}
```

Method C — REST endpoints (base = `http://127.0.0.1:8754`)

ACTION	CALL
Check in / heartbeat	POST /api/agents/update {id, status:"online", task, connectedVia:"api"}
Read a channel	GET /api/chat?channel=agent:sage
Reply to Lew	POST /api/chat {channel:"agent:sage", from:"sage", role:"agent", text}
Live event stream	GET /api/stream (Server-Sent Events)
Request approval	POST /api/approvals {subject, by:"sage", risk, detail}

CLI shortcuts (`bin/mc`)

```
mc set sage online --task "Routing the day" # turn the light green
mc say "On it, Lew." --as sage --channel agent:sage
mc read --channel agent:sage -n 20
```

Troubleshooting

SYMPTOM	FIX
Card says "Bridge degraded — CLI login needed"	Run <code>claude</code> → <code>/login</code> in Terminal. Heals within 10 min or rerun the launcher.
Reply says "(bridge: ... 401 / authenticate)"	Same — the CLI isn't logged in on this machine.
Agent went offline after closing Terminal	The launcher detaches processes, but a full logout/restart stops them. Rerun the launcher.
Two replies to one message	The template loop and a real bridge both answered. Always start via the launcher — it excludes bridged agents automatically.
"site can't be reached" on the dashboard	Backend isn't running. The launcher starts it; or <code>npm start</code> in the repo.

LOGS (WHEN IN DOUBT, READ THESE)

`/tmp/mc-8754.log` — backend · `/tmp/mc-agent-loops.log` — template loops · `/tmp/<id>-bridge.log` — that employee's bridge · CONFIDENTIAL